# Unit-1



# Algorithms & Problem Solving

Prepared By Mr. Vipin K. Wani

**Study material provided by: Vishwajeet Londhe**

## Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)




WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b




Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

# What is Algorithm

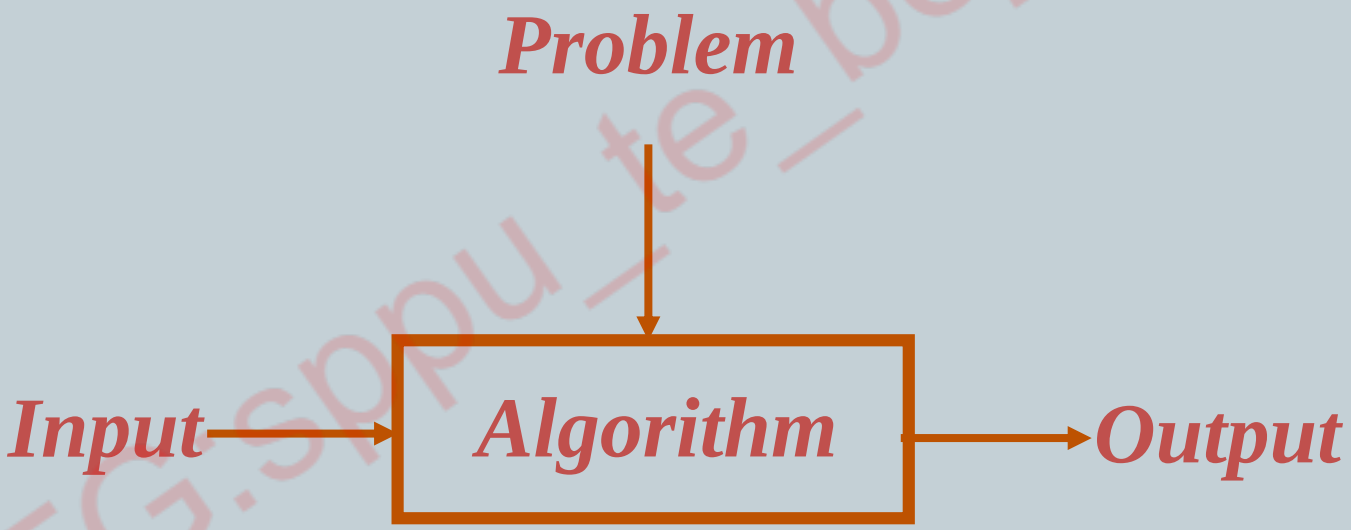


- **Algorithm is a set of rules for carrying out calculations either by hand or on machine.**
- **Algorithm is finite step by step procedure to achieve a required result.**
- **An algorithm is set of computational steps that transforms input into output.**
- **An algorithm is set of unambiguous instructions for solving a problem.**
- **A tool for solving a well-specified computational problem**

TG.sppu_te_be_comp

# What is Algorithm

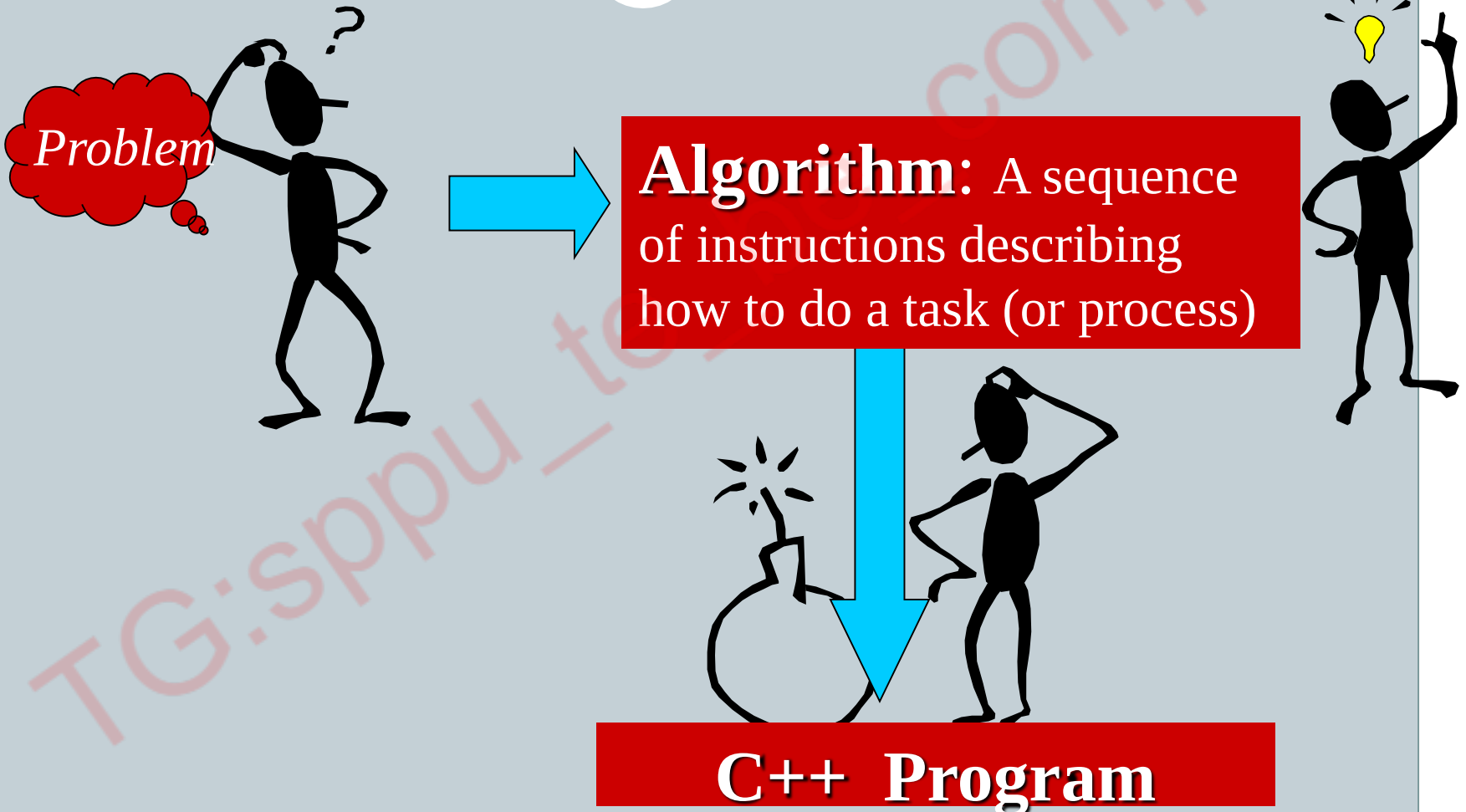





TG:sppu_te_be_comp

# What is Algorithm






TG:sppu_tech_comp

# Computational Problem



- A computational problem specifies an input-output relationship
- a computational problem is a problem that may be solved by an algorithm.

What does the input look like?

What should the output be for each input?

**Example: To find Prime number**

Input: an integer number n

Output: Is the number prime?

**Example: To sort a names of peoples in a list**

Input: A list of names of people

Output: The same list sorted alphabetically

TG:sppu_te_be_comp

# The Role of Algorithms in Computing



- An algorithm is a specific procedure for **solving a well-defined computational problem**.
- The development and analysis of algorithms is fundamental to all aspects (areas) of computer science.
- Like artificial intelligence, databases, graphics, networking, operating systems, security, and so on.
- Each step in algorithm give us a path to solve a particular problem.
- It maps from input to the output, in between these two there are multiple operations/steps.
- Programming is all about solving problem with the help of programming language. In order to write a program for a problem, the programmer must know the input and output and how to solve the problem. An algorithm solves the half of the above task. Algorithm can be written in simple in English sentences with clearly mentioning the steps. Otherwise, we can use pseudo code which is quite similar to the programming language key words.

TE_sppu_te_be_comp

# The Role of Algorithms in Computing



- An algorithm specifies what computation have to be performed to achieve a goal.
- An algorithm gives step by step guide to get a desired output.
- It also specifies expected input to be given to algorithm.
- It also specifies expected output.
- It also notify us for wrong navigation and wrong input.

TG:sppu_te_be_comp

# Characteristics of good algorithms



There can be many algorithms for a particular problem. So, how to classify an algorithm to be good and others to be bad?

**Correctness:** An algorithm is said to be correct if for every set of input it halts with the correct output. If you are not getting the correct output for any particular set of input,then your algorithm is wrong.

**Finiteness:** The algorithm must always terminate after a finite number of steps.

**Efficiency:** An efficient algorithm is always used. By the term efficiency, we mean to say that:

i. The computational time should be as less as possible.

ii. The memory used by the algorithm should also be as less as possible.

**Unambiguity**: Instructions written in a algorithms must be unambiguous

TG.sppu_te_be_comp

# Design & Analysis of Algorithm



Algorithm deals with two steps designing & analyzing the algorithms.

**The Design includes:**

- The description of an algorithm steps by means of pseudo language and Proof of correctness that is the algorithm solve the problem in all the cases.
- Algorithm design is all about the mathematical theory behind the design of good algorithms.

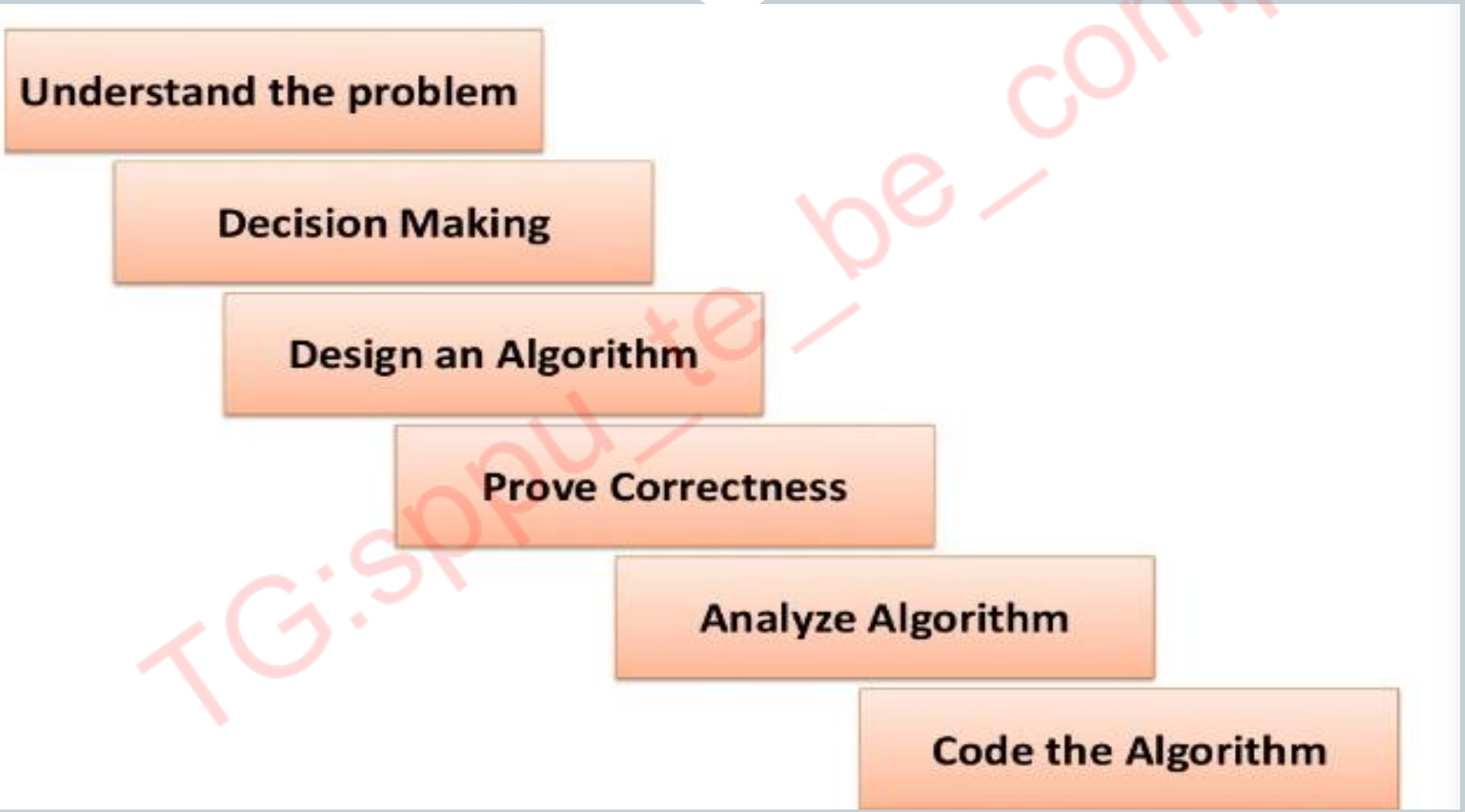
**The Analysis:**

- The Analysis deals with performance evaluation of an algorithm i.e. complexity analysis.
- Complexity is not dependent on machine or programing language.

TG:sppu_te_be_comp

# Algorithm Design Steps






TG:sppu_te_be_comp

# Why to analyze algorithm



Why to analyze algorithm:

- To Evaluate algorithm performance
- To Compare Different algorithms

What to analyze about them?

Running time, memory usage, Solution quality, Worst case performance etc.

TG:sppu_te_be_comp

# Components of an Algorithm



Various Components of an Algorithm are

- Variables and values
- Instructions
- Procedures
- Selections
  - An instruction that decides which of two possible sequences is executed
  - The decision is based on true/false condition
- Repetitions
  - Also known as iteration or loop
- Documentation
  - Records what the algorithm does

TG:spbu_fe_be_comp

# Example of an Algorithm



**Here the task is to write an algorithm to multiply 2 numbers and print the result:**

**Step 1:** Start
**Step 2:** Get the knowledge of input.
Here we need 3 variables; a and b will be the user input and c will hold the result.
**Step 3:** Declare a, b, c variables.
**Step 4:** Take input for a and b variable from the user.
**Step 5:** Know the problem and find the solution using operators, data structures and logic We need to multiply a and b variables so we use * operator and assign the result to c.
That is c <- a * b
**Step 6:** Here we need to print the output. So write print c
**Step 7:** End

TG.sppu_te_be_comp

# Classification of an Algorithm



The algorithms can be classified in various ways. They are:

1. Implementation Method
2. Design Method
3. Design Approaches
4. Other Classifications

TG:sppu_te_be_comp

# Classification of an Algorithm (Implementation Method)



**Classification by Implementation Method:**

**Recursion or Iteration:** A recursive algorithm is an algorithm which calls itself again and again until a base condition is achieved whereas iterative algorithms use loops to solve any problem.

**Example:** The Tower of Hanoi is implemented in a recursive fashion while Sorting algorithms problem is implemented iteratively.

**Exact or Approximate:** Algorithms that are capable of finding an optimal solution for any problem are known as the exact algorithm. For all those problems, where it is not possible to find the most optimized solution, an approximation algorithm is used. Approximate algorithms are the type of algorithms that find the result as an average outcome of sub outcomes to a problem.

**Example:** For NP-Hard Problems, approximation algorithms are used. Sorting algorithms are the exact algorithms.

TG:sppu_te_be_comp

# Classification of an Algorithm (Implementation Method)



**Serial & Parallel or Distributed Algorithms:** In serial algorithms, one instruction is executed at a time while parallel algorithms are those in which we divide the problem into sub problems and execute them on different processors. If parallel algorithms are distributed on different machines, then they are known as distributed algorithms.

TG:sppu_te_be_comp

# Classification of an Algorithm (Design Method)



**Greedy Method:** In the greedy method, at each step, a decision is made to choose the *local optimum*, without thinking about the future consequences.
**Example:** Fractional Knapsack, Activity Selection.
**Divide and Conquer:** The Divide and Conquer strategy involves dividing the problem into sub-problem, recursively solving them, and then recombining them for the final answer.
**Example:** Merge sort, Quicksort.

**Dynamic Programming:** The approach of Dynamic programming is similar to divide and conquer. The difference is that whenever we have recursive function calls with the same result, instead of calling them again we try to store the result in a data structure in the form of a table and retrieve the results from the table. Thus, the overall time complexity is reduced. "Dynamic" means we dynamically decide, whether to call a function or retrieve values from the table.
**Example:** 0-1 Knapsack, subset-sum problem.

TE-sppu_te_it_comp

# Classification of an Algorithm (Design Method)



**Backtracking:** This approach is based on exploring each available option at every stage one-by-one. While exploring an option if a point is reached that doesn't seem to lead to the solution, the program control backtracks one step, and starts exploring the next option. In this way, the program explores all possible course of actions and finds the route that leads to the solution.
**Example:** N-queen problem, maize problem.

**Branch and Bound:** This technique is very useful in solving combinatorial optimization problem that have *multiple solutions* and we are interested in find the most optimum solution. In this approach, the entire solution space is represented in the form of a state space tree. As the program progresses each state combination is explored, and the previous solution is replaced by new one if it is not the optimal than the current solution.
**Example:** Job sequencing, Travelling salesman problem.

TGtspu_te_be_comp

# Classification of an Algorithm (Design Approaches)



**Top-Down Approach:** In the top-down approach, a large problem is divided into small sub-problem. and keep repeating the process of decomposing problems until the complex problem is solved.

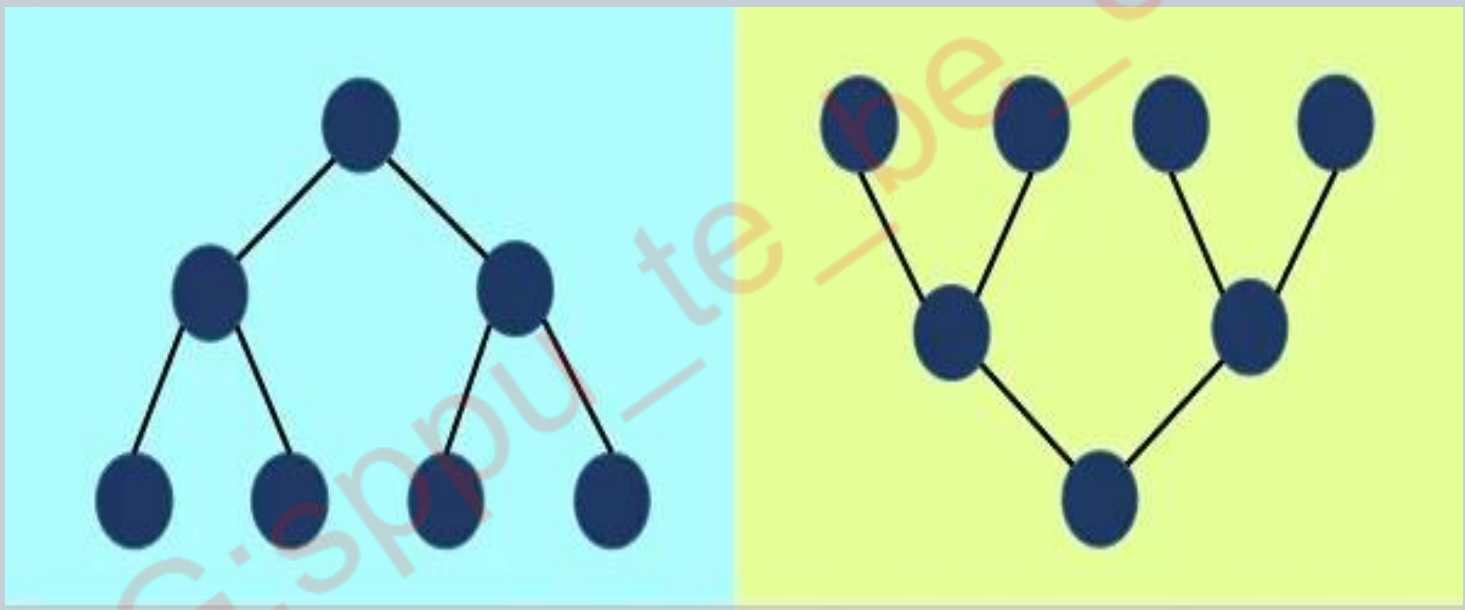
**Bottom-up approach:** The bottom-up approach is also known as the reverse of top-down approaches.

In approach different, part of a complex program is solved using a programming language and then this is combined into a complete program.

TG:sppu_te_be_comp

# Classification of an Algorithm (Design Approaches)






TG:spou_te_be_comp

# Classification of an Algorithm (Other Approaches)



algorithm can be classified into other broad categories like:

**Classification by complexity:**

**Classification by Research Area:**

Etc…

TG:sppu_te_be_comp

# Algorithm as technology



an algorithm usually refers to **a small procedure that solves a recurrent problem.** Algorithms are also used as specifications for performing data processing and play a major role in automated systems.

Total system performance depends on choosing efficient algorithms as much as on choosing fast hardware. Just as rapid advances are being made in other computer technologies, they are being made in algorithms as well.

TG:sppu_te_be_comp

# Algorithm as technology



Computer A

Faster
Can Execute 10000 Instructions /Sec
100 times Faster than B
i.e. $10^4$ /sec

Computer B

Slower
Can Execute 100 Instructions /Sec
100 times Slower than A
i.e. $10^2$ /sec

Consider we are applying Insertion Sort algorithm on Computer A with time complexity C1 $n^2$ and
we are applying Marge Sort algorithm on Computer B with time complexity C2nlogn

SO time taken to execute Algorithm for 100 input will be

$T(A) = C1n^2 / 10^4 = 2 * (10^2)^2 / 10^4 = 2$

$T(B) = C2\ n\log n / 10^2 = 2 * (10^2) \log (10^2) / 10^2 = 4$



TG:spu_te_be_comp

# Algorithm as technology



SO time taken to execute Algorithm for 10000 input will be

$T(A) = C1n^2 / 10^4 = 2 * (10^4)^2 / 10^4 = 20000$ sec

$T(B) = C2\ n\log n / 10^2 = 2 * (10^4) \log (10^4) / 10^2 =$ 800 Sec

The Reason is multiplier

$T(A) = C1\ n * n$

$T(B) = C2\ n \log n$

TG:sppu_te_be_comp

# Algorithm as technology



- So Designing of proper algorithm is important.
- This huge difference is because of the logic technology used in a algorithm.
- Algorithm must be effective

TG:sppu_te_be_comp

# Evolution of Algorithm



- Ada Lovelace, an English mathematician and daughter of the poet Lord Byron, wrote the first algorithm for a machine in the 1800s and is considered the first computer programmer.

TG:sppu_te_be_comp

# Correctness of Algorithm



- An algorithm Is called totally correct for the given specification and only if for any correct input data:
- Stops and
- Returns correct output

TG:sppu_te_be_comp

# Correctness of Algorithm



- The only way to prove the correctness of an algorithm over all possible inputs is **by reasoning formally or mathematically about it**.
- Proving correctness of algorithm is crucial. For many problems, algorithms are very complex.
- Reliability and efficiency of an algorithm cannot be claimed unless and until it gives the correct output for each of the valid inputs.
- The correctness of an algorithm can be quickly proved by checking certain conditions.

TG:sppu_te_be_comp

# Correctness of Algorithm



**Proving Correctness How to prove that an algorithm is correct?**

- Proof by:
- Counterexample (indirect proof )
- Induction (direct proof )
- Loop Invariant
- proof by contradiction
- proof by contrapositive

For any algorithm, we must prove that it always returns the desired output for all legal instances of the problem.

For sorting, this means even if the input is already sorted or it contains repeated elements.

TG:sppu_te_be_comp

# Correctness of Algorithm



**Proof by Counterexample** Searching for counterexamples is the best way to disprove the correctness of some things.

- Counter example is an single example that proof the statement is false.
- Here single example is Sufficient to prove
- Identify a case for which something is NOT true

Ex 4n +n is always multiple of 8

consider n=2

So 4(2)+2 =10 is not multiple of 8

Ex2. $n^2$ is always even if n is even.

TG:sppu_to_be_comp

# Correctness of Algorithm



**Proof By Induction:**

Mathematical induction is a very useful method for proving the correctness of recursive algorithms.

1. Prove base case
2. Assume true for arbitrary value n
3. Prove true for case n + 1 Proof by Loop Invariant

Example Let us prove sum of integers from 1 to n = n(n+1)/2

**Step 1: Prove for Base case**

For n=1

N(1) = 1(1+1)/2 =2/2=1 is True

**Step 2(Hypothesis ): Assume true for arbitrary value k i.e for n=k k(k+1)/2 is true**

**i.e. 1 +2 + 3+ ……..+ K = k(k+1)/2 ----------1**

TG:sppu_te_be_comp

# Correctness of Algorithm



**Step 3: Now prove for induction step i.e for n=k+1**

**So for n=k+1**

**n(k+1) =1+ 2 +3+…….+ K+ K+1 = (k+1)((k+1)+1)/2**

**= (k+1)(k+2)/2**

**Consider LHS**

**N(k+1) =1+ 2 +3+…….+ K+ K+1**

**= k(k+1)/2 + (k+1) …….from Step 2**

**= k(k+1)/2 + 2(k+1) /2**

**=(k+1)(k+2)/2**

**=RHS**

**So which is equivalent to n(n+1)/2 AS N=K+1 is prooved**

TG:sppu_te_ioe_comp

# Correctness of Algorithm



**Proof by Contradiction:**
Is a proof technique in which we assume given statement is true and then by logical solution we come to know that assumed statement is false.

**Example 1:** Prove the following statement by contradiction:
The sum of two even numbers is always even.
Assume the sentence The sum of two even numbers is always even is false.
Lets Check 2+2 =4 6+ 4 =10 2+8 =10 hence contradiction & assumption is wrong.

**Example 2:** Prove the following statement by contradiction:
The sum of two positive numbers is always positive.
Assume the sentence sum of two positive numbers is always positive is false.
Let us check
2+2=4
5+7 =12
It shows a contradiction hence the sentence sum of two positive numbers is always positive is **True**.

TG.sppu_te_be_comp

# Correctness of Algorithm



**Proof by Contrapositive:**

Here if we want to prove P→ Q is true then we will try to prove ~p→ ~q is true

**Example 1: If $n^2$ is even then n is even**

**Try to prove**

**If n is not even then $n^2$ is not even**

**Example if n=3 then n*n = 9 is not even and which is contrapositive.**

TG:sppu_te_be_comp

# Correctness of Algorithm



➢ **Loop Invariant approaches:**

• Useful for algorithms that loop. Formally: find loop invariant, then prove:

1. Define a Loop Invariant
1. Initialization
2. Maintenance
3. Termination

**Here we try to prove:**

**Initialization:** Conditions true before the first iteration of the loop

**Maintenance:** If the condition is true before the loop, it must be true before the next iteration.

**Termination:** On termination of the loop, invariant gives us a useful property that helps us to prove the correctness of the algorithm.

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Use of Loops
  - Writing initialization
  - Proper condition that will terminate after some iteration
- Efficiency of Algorithm

TG:sppu_te_be_comp

# Iterative algorithm design issues



- The correct use of loops in programs
- Factors that affect the efficiency of algorithms
- How to estimate and specify execution times:
  - Difficult to estimate and specify execution times
- How to compare algorithms in terms of their efficiency
  - Difficult to compare algorithms in terms of their efficiency

TG:sppu_te_be_comp

# Iterative algorithm design issues



- The correct use of loops in programs
  - Initial condition: Starting Condition i=0.

  Ex. i=0;

  - Termination: Condition that cause to terminate loop

    For i=0 to 10 do something……..endfor

    For i=m to n do something ……..endfor

    For i>0 and i<10 do something ……..endfor…….. **Not a Proper termination**

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm
  - Use of resources
  - CPU time and internal memory
  - **There are four way to improve efficiency**
    1. Remove Redundant computation Outside the loop
    2. Referencing Array Elements
    3. Inefficiency due to late termination
    4. Early detection of desired output condition

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm
  1. **Remove Redundant computation Outside the loop**

- **Example: To calculate X & y**

- **Version 1:**

```
Int y, x=0;
For i=1 to n do
Begin
        int a, b;
        x=x+0.02
        y=(a*a*a+c)*x*x+b*b*x;
        Print (X & Y)
end
```

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm
  1. **Remove Redundant computation Outside the loop**

- **Example: To calculate X & y**

- **Version 2:**

```
Int y, x=0, a, b;
For i=1 to n do
Begin
        x=x+0.02
        //y=(a*a*a+c)*x*x+b*b*x;
        y=(a^3+c)*x^2+xb^2
        Print (X & Y)
end
```

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm

**2. Referencing Array Elements**

- **Finding Max element from array**

**Version 1:**

```
Int p=0, i=0;
For i=1 to n do
Begin
        if(a[i] > a[p])
        max=a[i];
        p=i;
end
```

**Version 2:**

```
Int p=0, i=0, max=a[i];
For i=1 to n do
Begin
        if(a[i] >max)
        max=a[i];
        p=i;
end
```

TGtspu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm

**3. Inefficiency due to late termination**

- **Example: Searching a roll_no in alphabetically sorted list.**

**Version 1:**

**While roll_no == current_rollno & ! EOF do get names from list.**

**Version 2:**

**While name roll_no > current_rollno & ! EOF do get names from list.**
**if current_rollno == roll_no**

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm

**4. Early detection of desired output condition**

- **Example of bubble sort**

**Version 1:**

```
Int i=n;
For i=1 to n-1 do
For j=1 to 1 do
Begin
        if currkey>nextkey
                exchange the item
        endif
end
```

TG:sppu_te_be_comp

# Iterative algorithm design issues



- Efficiency of algorithm

**4. Early detection of desired output condition**

- **Example of bubble sort**

**Version 2:**

```
Int i=n;
For i=1 to n-1 do
For j=1 to n-1 do
Begin
        if currkey>nextkey
                exchange the item
        endif
        if no exchange done
                retuen
        else
                i=i+1;
end
```

TG:sppu_te_be_comp

# What is Problem?



- A problem is **a situation, question, or thing that causes difficulty, stress, or doubt**.
- A problem is also a question raised to inspire thought.

TG:sppu_te_be_comp

# What are types of Problem?



- There are two types of problems:

  1. Problems based on algorithmic solutions

  2. Problems based on heuristic solutions

TG:sppu_te_be_comp

# What are types of Problem?



- **1. Problems based on algorithmic solutions**

  a. Algorithm is the sequence of instructions used to solve some problem.

  b. For solving some problem, series of actions are taken to reach to the solution.

  c. Example: problem of 'Making a Cup of Tea' can be solved by following the procedure(Series of actions)

TG:sppu_te_be_comp

# What are types of Problem?



- **2. Problems based on heuristic solutions:**
- a. There are some problems that cannot be solved by just following certain actions.
- b. In solving such problems, critical decisions has to be made. We need to follow some trial and error.
- c. The solutions that cannot be reached through direct set of steps are called heuristic solutions.
- d. Example: the problem of "Which stock should I buy" is based on heuristic solution as it requires, knowledge, experience of trials and errors, skills, careful analysis of market and so on.
- e. With heuristic solution problem solver has to follow six steps of problem solving more than once.

TG.sppu_te_be_comp

# What are types of problem solving strategies?



- Divide and Conquer
- Dynamic programming
- Greedy Search
- Backtracking
- Branch and Bound
- Algorithm
- Trial and Error
- Solve by analogy
- The building block approach

TG.sppu_te_be_comp

# What are types of problem solving strategie?



- **Divide and Conquer:** The whole problem is divided into two subproblems. If sub-problems are large then these are divided further into sub sub-problem. Then these sub problem solve independently. All the solutions from all the sub problems are collected to get final solution.
- **Dynamic programming:** This method is used when we have to build up a solution to the problem via sequence of intermediate steps. Example: Fibonacci series.
- **Greedy Search**: Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.
- **Backtracking:** A method of solving combinatorial problems by means of an algorithm which is allowed to run forward until a dead end is reached, at which point previous steps are retraced and the algorithm is allowed to run forward again.

TG:sppu_te_be_comp

# What are types of problem solving strategie?



- **Branch and Bound**: Branching is the process of spawning sub problems, and bounding refers to ignoring partial solutions that cannot be better than the current best solution.
- **Algorithm:** The step-by- step procedure (instructions) for performing particular task. The instructions are nothing but the statements in simple English language.
- **Trial and Error:** It is approach that deals with trying a number of different solutions and ruling out the one that do not work.
- **Brain storming technique:** It is group activity. Group members are gathered together to discuss a problem and give solutions upon it.
- **Solve by analogy:** The idea behind this technique is that it looks for existing solution for similar type of problem and apply it on the given problem.
- **The building block approach:** It combines the idea of solving by analogy and divide and conquer techniques. It checks if any solutions for smaller pieces of the problem exist.

TC:sppu_te_be_comp

# Classification of time Complexities



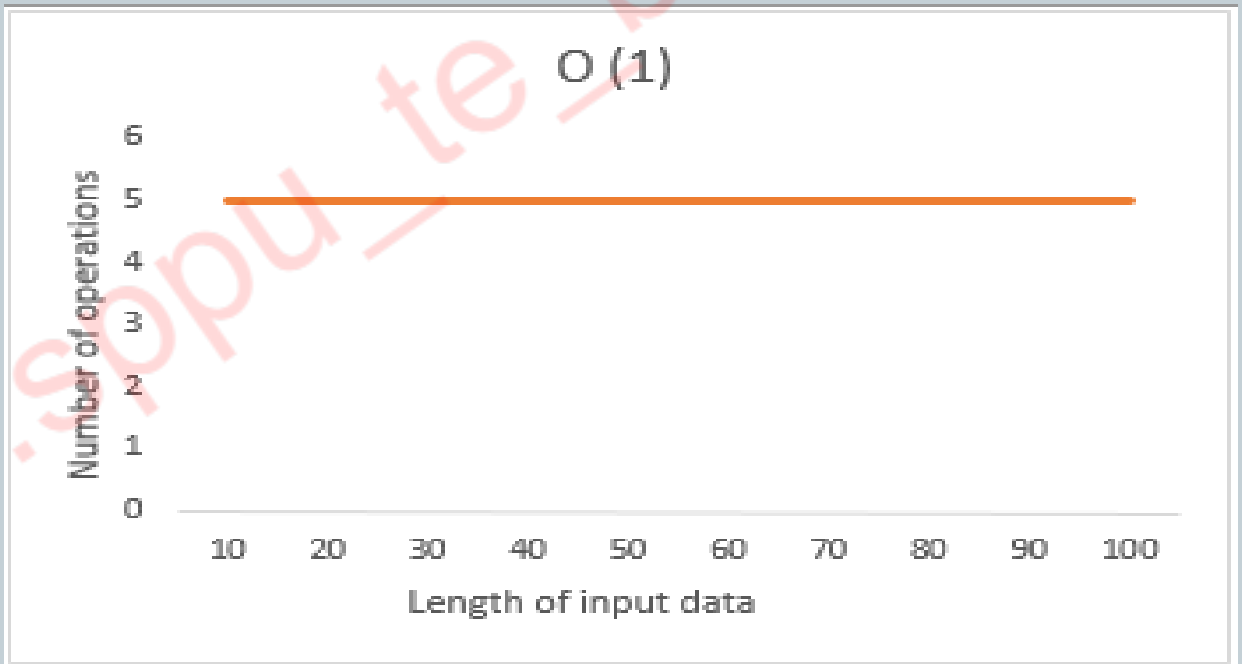
- **Time complexities are classified based on time taken to complete the no of operations for n number of input.**
- **Various types are:**
- Constant Time Complexity: O(1)
- Linear Time Complexity: O(n)
- Logarithmic Time Complexity: O(log n)
- Quadratic Time Complexity: $O(n^2)$
- Exponential Time Complexity: O(2^n)
- Cubic time – O (n^3)

TC-sppu_te_be_comp

# Classification of time Complexities



- **Constant Time Complexity O(1):**
- An algorithm is said to have constant time with order O (1) when it is not dependent on the input size n. Irrespective of the input size n, the runtime will always be the same.
- **Ex: c=a+b;**



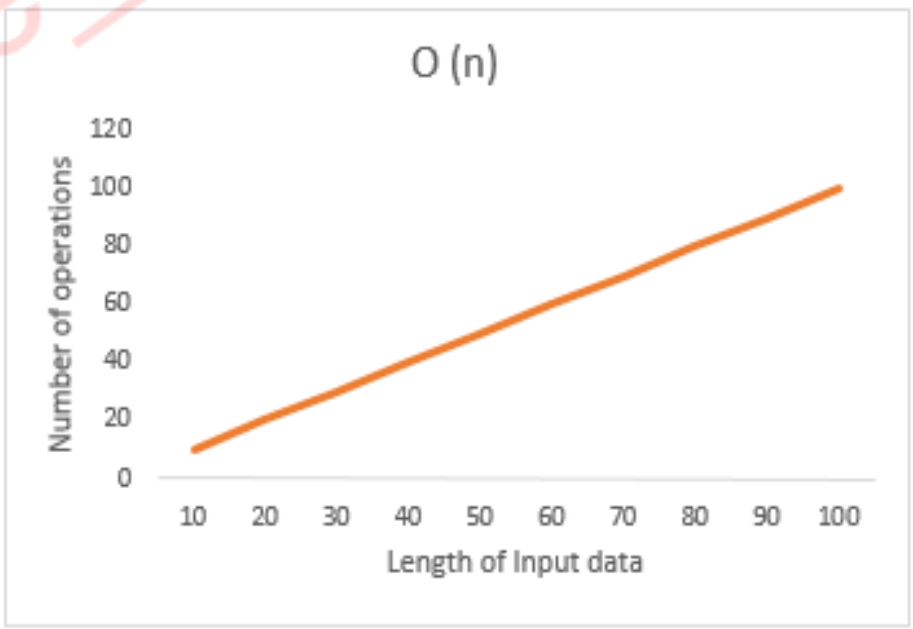

TG:sppu_te_be_comp

# Classification of time Complexities



- **Linear Time Complexity: O(n)**
- An algorithm is said to have a linear time complexity when the running time increases linearly with the length of the input. When the function involves checking all the values in input data, with this order O(n).
- For example:

```
for (i=0; i<n; i++)
 {
        Print("Hello");
 }
```



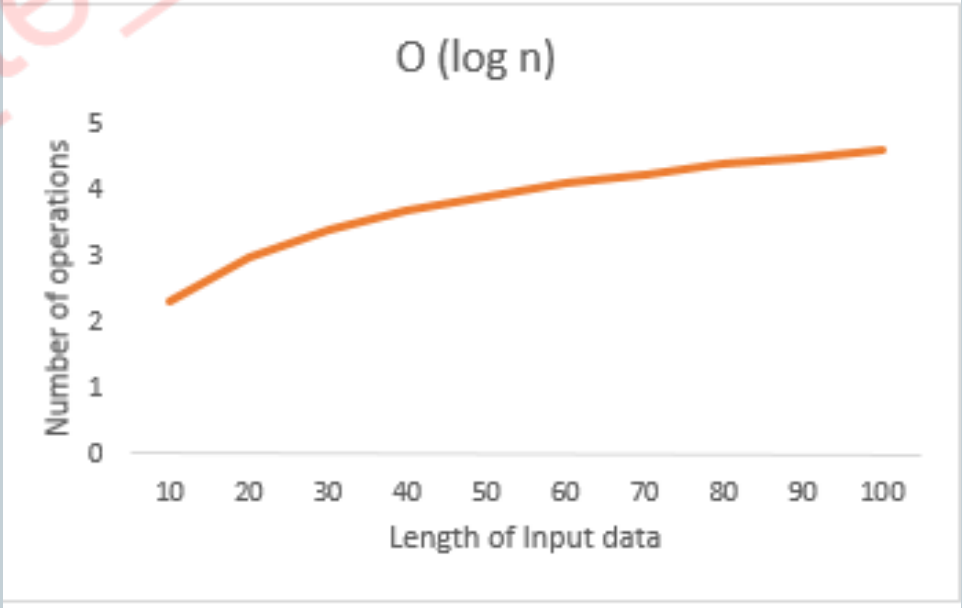

TG:sppu_te_be_comp

# Classification of time Complexities



- **Logarithmic Time Complexity: O(log n)**
- An algorithm is said to have a logarithmic time complexity when it reduces the size of the input data in each step. This indicates that the number of operations is not the same as the input size. The number of operations gets reduced as the input size increases.
- For example:

1. Binary search
2. Marge Sort, Quick sort



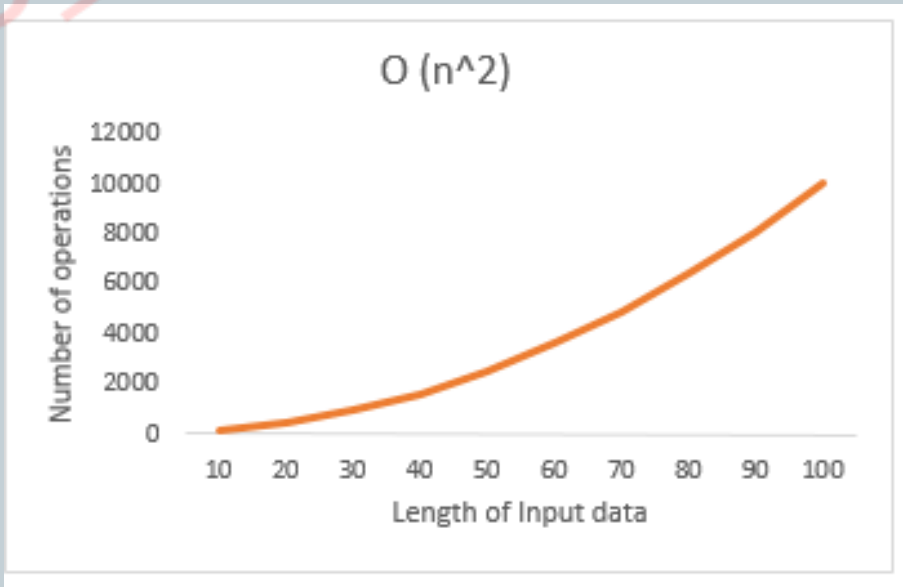

TG:sppu_te_be_comp

# Classification of time Complexities



- **Quadratic Time Complexity: $O(n^2)$**
- An algorithm is said to have a non – linear time complexity where the running time increases non-linearly (n^2) with the length of the input.
- In this type of algorithms, the time it takes to run grows directly proportional to the square of the size of the input.
- For example:

1. Bubble Sort, Insertion sort



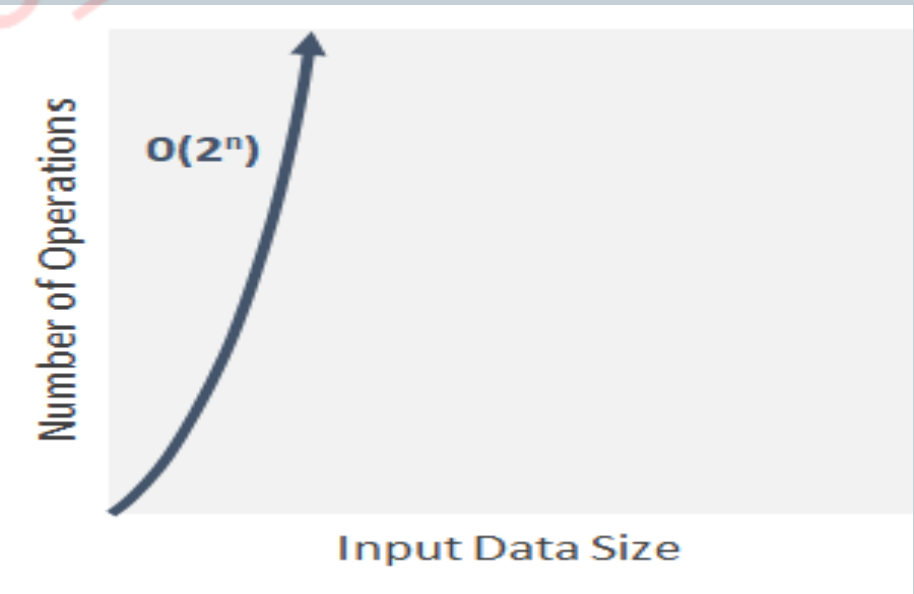

TG:sppu_te_be_comp

# Classification of time Complexities



- **Exponential Time Complexity: O(2^n)**
- In exponential time algorithms, the growth rate doubles with each addition to the input (n), often iterating through all subsets of the input elements. Any time an input unit increases by 1, it causes you to double the number of operations performed. For example:

1. Brute-Force algorithms




TG:sppu_te_be_comp

# Classification of time Complexities

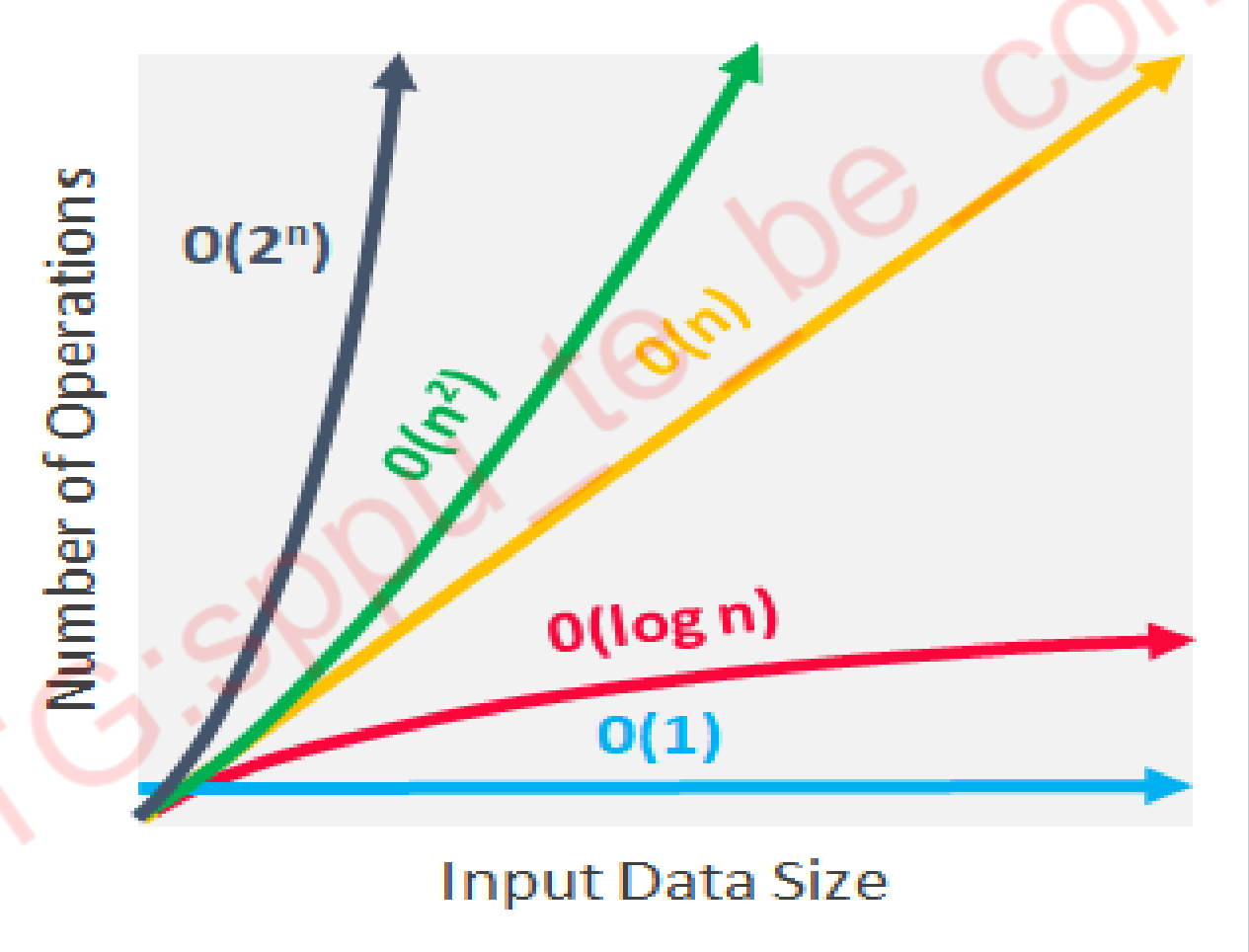


- **Cubic time – O (n^3)**
- In quadratic algorithm, we used two nested loops. In cubic algorithm we use three nested loops.
- An algorithm is said to run in cubic time if the running time of the three loops is proportional to the cube of N
- Example: Multiplication of matrices

TG:sppu_te_be_comp

# Classification of time Complexities






TG:spot_to be comp

# Exercise



- What is algorithm? Describe the role of algorithms in computation.
- Enlist the steps to design any algorithm. Explain each step in detail.
- Explain various approaches to classify the algorithms.
- With a suitable example prove that algorithm is a technology.
- What is correctness of algorithm? Describe the need of correctness.
- Explain various techniques to prove correctness of algorithm.
- Explain some of the important factors used to analyze the algorithms.
- Discuss various design issues of iterative algorithm.
- Write detailed note on classification of time complexities.
- What is problem? What are types of problem?
- Explain the various problem solving strategies.

TG:sppu_te_be_comp

# Thank You…!






TG:sppu_te_comp